中国大学生计算机设计大赛



软件开发类作品文档

作品编号：2025012245

作品名称：儿童绘本馆智慧运营管理系统

作　　者：柏佳慧 焦新亨 陈诚 刘文玲 姚思宇

版本编号：V1.0

填写说明：

1、本文档适用于所有涉及软件应用与开发类的各个小类作品，包括：（1）Web 应用与开发（2）管理信息系统（3）移动应用开发（非游戏类）（4）算法设计与应用（5）企业赛项；

2、本文档为简要文档，不宜长篇大论，需简明扼要，建议设计二级目录，逻辑性强；

3、一级标题采用二号黑体，居中，二级标题采用三号黑体，靠左，根据需要可以设计三级标题，正文一律用五号宋体；

4、提交文档时，以 PDF 格式提交本文档；

5、本文档内容是正式参赛内容组成部分，务必真实填写。如不属实，将导致奖项等级降低甚至终止本作品参加比赛。

6、项目中涉及应用人工智能大模型的内容，务必在各个部分说明清楚。

填写日期：2025 年 5 月 4 日

目录

第一章 需求分析 3

1.1 项目背景 3
1.2 项目目标 3
1.3 系统亮点与创新点 3

第二章 概要设计 5

2.1 设计方案 5
2.2 系统设计 5

第三章 详细设计 10

3.1 界面设计 10
3.2 数据库设计 14
3.3 关键代码 15

第四章 测试报告 22

第五章 安装及使用 26

5.1 安装环境 26
5.2 安装过程 26
5.3 主要流程： 26

第六章 项目总结 27

6.1 项目协调 27
6.2 任务分解 27
6.3 克服困难 27
6.4 升级演进 27

参考文献 28

第一章需求分析

1.1 项目背景

当前我国 0-12 岁儿童数量超过 2 亿，儿童早期阅读推广成为教育体系的重要组成部分。同时国家政策利好，2025 年 3 月国务院法制办公室提出《全民阅读促进条例（征求意见稿）》，为绘本行业指明新的发展方向，鼓励绘本行业不断社区化、全民化、数字化发展，完善的法律法规保障绘本馆的持续蓬勃发展。各地绘本馆数量迅速增长，但传统绘本馆在绘本管理、会员服务、活动组织、志愿者协作等环节普遍存在管理分散、数据孤立、服务效率低下的问题。调研显示，超 65%的小型绘本馆在运营过程中面临库存混乱、会员粘性不足、活动反馈不及时等难题，急需一套集成化、智能化、可扩展的系统支撑运营优化与决策提升。

1.2 项目目标

本系统通过模块化设计，全面打通绘本馆从绘本采购、库存管理到借阅归还、会员积分、志愿者协作与活动反馈的全链条流程，实现了数据的集中管理与业务的自动化处理。系统采用高并发支持架构，确保在绘本高峰借阅期依然保持流畅体验，并结合实时数据分析为绘本馆管理者提供精准决策依据。同时，系统引入灵活的激励机制，有效提升会员与志愿者的参与度与粘性，助力绘本馆运营水平持续提升。我们开发一套面向儿童绘本馆智慧管理需求的系统平台，旨在覆盖绘本采购、借阅、归还、捐赠、库存预警、会员激励、志愿者管理、活动闭环运营等核心业务流程，结合实时数据分析与可视化决策支持，提升绘本馆资源使用率、会员活跃度与整体服务质量，有效赋能教育、文化、商业、公共服务等领域的阅读项目，实现儿童阅读环境的可持续优化发展。



图 1 各领域应用

1.3 系统亮点与创新点

(1) 绘本资源全生命周期管理

系统覆盖绘本从采购、入库、借阅、归还到捐赠的全过程管理，支持批量数据导入与多条件检索查询，并通过库存预警机制智能提示库存短缺，帮助绘本馆动态掌控资源状况，优化采购决策。

(2) 智能化数据分析与可视化决策支持

在首页数据看板中，系统以折线图、饼图、柱状图等形式实时展示绘本借阅趋势、热门绘本排行、会员活跃度、库存变化等核心指标，辅助管理者快速发现问题与机遇，提升运营决策的科学性与时效性。



图 2 首页页面

(3) 会员积分体系与志愿者激励机制

通过积分累积、等级划分与称号授予，系统激励会员频繁借阅、积极参与活动，提升粘性；志愿者服务记录与贡献排名功能，结合积分与荣誉体系，进一步激发志愿者参与热情，形成自驱动良性循环。

(4) 活动闭环管理体系

系统支持活动从创建、宣传、报名、执行到反馈收集与数据分析的全流程管理，自动统计活动参与人数、完成率、满意度等指标，帮助绘本馆不断优化活动设计与资源投入，提升运营效率和活动影响力。

(5) 高效可靠的系统架构设计

前端采用Next.js与Ant Design框架，界面清新，响应迅速；后端基于Golang Gin框架，配合MySQL数据库与Redis缓存，确保高并发访问下系统依然流畅稳定。Docker容器化部署实现快速上线与弹性扩展，支持多环境无缝迁移与维护，保障系统的可持续发展能力。

(6) 符合儿童体验的人性化交互设计

界面采用暖色调与亲和式布局设计，简化操作流程，符合儿童阅读空间温馨、舒适的氛围，提升读者与工作人员的整体体验感。

(7) 扩展性强，支持未来升级

系统设计预留接口，未来可无缝集成智能推荐模块（基于借阅行为推荐绘本）、智能客服模块等，持续赋能绘本馆智慧化运营发展。

第二章概要设计

2.1 设计方案

本项目以提升绘本馆运营效率和服务质量为核心目标，围绕绘本馆业务流程进行模块化设计，实现数据集中管理与业务自动化处理。利用数据分析为运营决策提供支持，并引入激励机制提高会员和志愿者参与度，致力于打造一个注重用户体验、温馨、便捷的交互网站。

2.2 系统设计

2.2.1 系统功能模块及其层次结构

一级界面：系统的核心功能入口界面，包含【首页】、【库存管理】、【会员管理】、【活动管理】、【志愿者管理】、【绘本管理】六大模块。以首页为例，其左侧导航栏集成各模块入口，方便用户快速切换；中间展示关键运营数据，如绘本总数、借阅量等，助用户了解整体运营状况；消息通知区集中呈现系统通知与提醒，保障运营顺畅。这些模块涵盖绘本馆运营各关键环节，用户能在此进行基本的管理操作与信息查看。

二级界面：是一级界面各模块的细分功能操作界面。在库存管理模块中，信息检索区可按多种条件检索绘本库存；操作功能区支持新增记录、批量导入等操作；库存信息展示区以表格呈现库存详情并提供编辑删除功能。会员管理模块的二级界面包括信息检索、会员信息操作和会员统计等区域，实现会员信息精准管理与分析。二级界面细化了一级界面功能，满足用户更深入的操作需求。

三级界面：是对二级界面特定功能的进一步拓展和细化。在志愿者管理的志愿者招募模块中，筛选与搜索区提供多种条件查找招募活动；招募活动管理区可创建、编辑、查看和删除招募活动；志愿活动审核区用于审核活动，确保活动规范合理。绘本借阅模块的借阅历史记录按不同状态分类展示，还提供详情查看与快捷操作；统计分析区生成不同类型绘本借阅数量柱状图，为采购管理提供数据支持。三级界面针对具体业务场景，提供深度操作与数据分析功能。

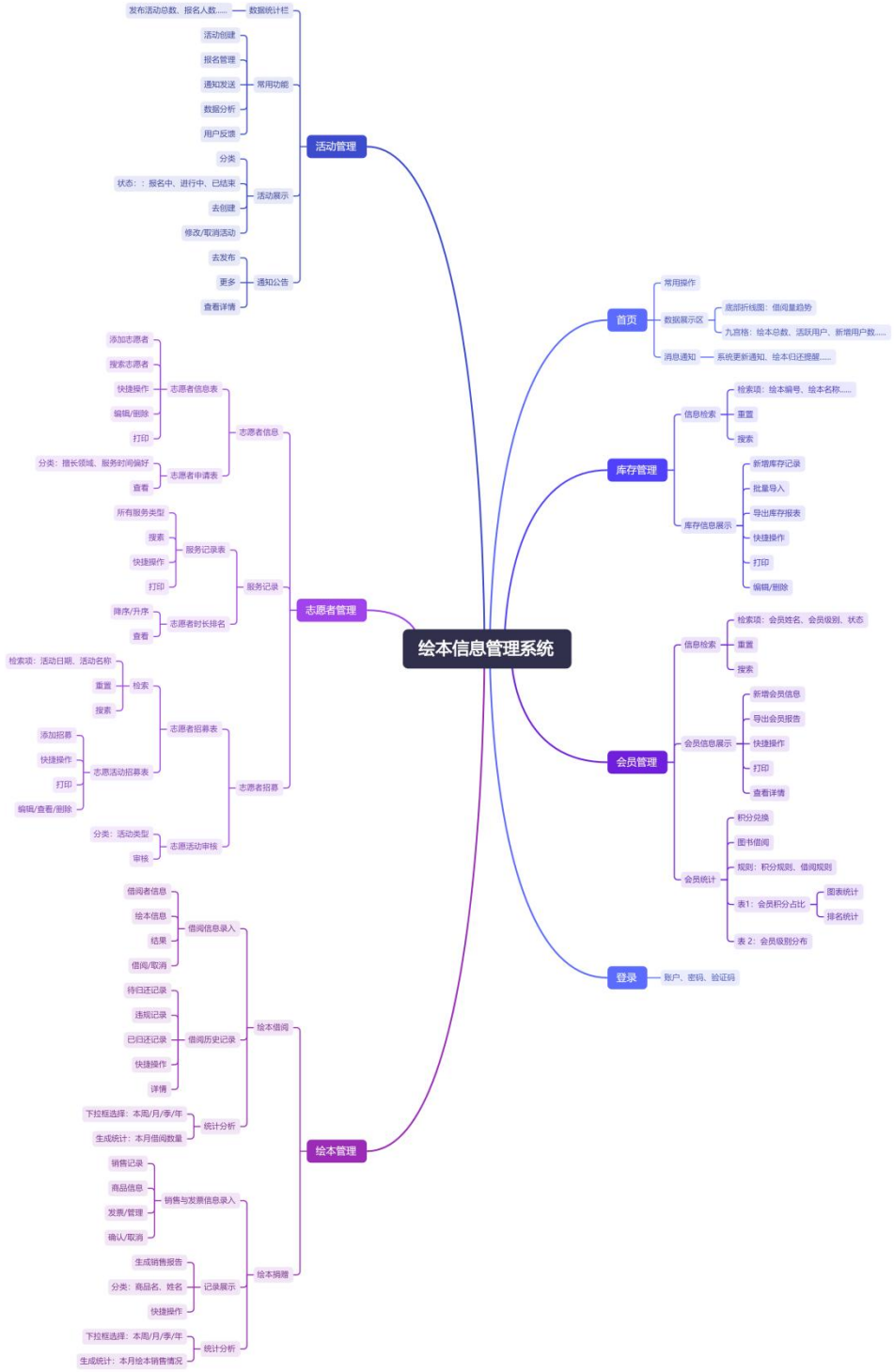


图 3 层次结构图

2.2.2 后端技术框架

后端技术框架围绕 Golang 展开，结合 Gin 框架构建 API 服务，使用 GORM 进行数据库操作，Redis 实现缓存功能，同时利用日志库记录运行时信息，形成一套高效稳定的后端架构体系。

编程语言：选用 Golang，凭借其高效的并发性能、简洁的语法和丰富的标准库，满足系统在处理多用户请求、数据存储与业务逻辑处理方面的需求。

Web 框架：采用 Gin 框架，具有高效的路由系统、中间件支持等特性，能快速构建稳定

的 API 服务。在系统中，Gin 框架用于处理 HTTP 请求和响应，如处理用户登录请求的 Login 函数，通过 Gin 框架解析请求参数、进行业务逻辑处理，并返回相应结果。

数据库操作：使用 GORM 作为数据库操作工具，对主流数据库（如 MySQL、PostgreSQL 等）提供了良好的支持。通过 GORM，开发者可以方便地进行数据库表的创建、数据的查询、插入、更新和删除等操作。在系统中，通过定义不同的数据访问对象，同时利用 GORM 实现对绘本借阅记录、库存信息等数据的持久化管理。

缓存机制：引入 Redis 作为缓存工具，用于缓存数据以提高系统的响应速度和性能。在系统中，Redis 被用于缓存绘本信息、库存信息、借阅统计数据等。

日志管理：使用日志库（如代码中的 logger.LogPrinter）进行系统运行时的日志记录。日志记录对于系统的调试、监控和故障排查非常重要，它可以记录系统运行过程中的关键事件、错误信息等。在该系统中，通过日志库记录如缓存保存失败、获取数据失败等异常情况，方便开发者及时发现和解决问题。

2.2.3 模块调用关系

首页模块：通过左侧导航栏子模块调用其他功能模块，如点击库存管理按钮进入库存管理模块；常用操作与数据展示子模块调用库存管理、会员管理、绘本管理等模块的数据进行展示；消息通知子模块接收其他模块产生的通知消息，如活动管理模块的活动通知、绘本管理模块的借阅提醒等。

库存管理模块：信息检索子模块为操作功能子模块和库存信息展示子模块提供检索服务；操作功能子模块对库存信息展示子模块中的数据进行添加、修改、删除等操作，并可调用导出库存报表功能。

会员管理模块：信息检索子模块为会员信息操作子模块和会员统计子模块提供检索服务；会员信息操作子模块可对会员信息进行新增、编辑等操作，会员统计子模块调用会员信息操作子模块的数据进行统计分析和展示。

活动管理模块：数据统计子模块从常用功能子模块、活动展示子模块获取数据进行统计分析；常用功能子模块中的活动创建、报名管理等功能会影响活动展示子模块的活动状态和信息；通知公告子模块用于发布活动相关通知。

志愿者管理模块：志愿者信息子模块、服务记录子模块、志愿者招募子模块之间相互独立又存在数据关联，如志愿者招募子模块审核通过的志愿者信息可添加到志愿者信息子模块，志愿者服务记录可记录在服务记录子模块。

绘本管理模块：绘本借阅子模块和绘本捐赠子模块相互独立，绘本借阅子模块的借阅历史记录和统计分析数据为管理者提供决策支持；绘本捐赠子模块的捐赠信息展示和排名统计可用于激励捐赠人。

2.2.4 模块间接口

模块名称	接口名称	请求方式	接口地址
登录	登录	POST	/api/v1/auth/login
登录	获取图片验证码	GET	/api/v1/auth/get_verification_code
用户	查询用户	GET	/api/v1/user/search

模块名称	接口名称	请求方式	接口地址
用户	获取会员的统计数据	GET	/api/v1/user/vip_statics
借书	借书	POST	/api/v1/book/borrow/add
借书	获取可借用的书籍	GET	/api/v1/book/borrow/get_available
借书	查询借书记录	GET	/api/v1/book/borrow/query
借书	获取统计借阅记录	GET	/api/v1/book/borrow/query_statistics
借书	更新借阅状态	PUT	/api/v1/book/borrow/update_status
库存	添加库存	POST	/api/v1/book/stock/add
库存	模糊查询库存信息	GET	/api/v1/book/stock/fuzzy_query
捐赠	获取捐赠排名	GET	/api/v1/book/donate/get_ranking
捐赠	列出捐赠记录	GET	/api/v1/book/donate/list
活动管理	新增活动	POST	/api/v1/activity/add
活动管理	获取活动统计信息	GET	/api/v1/activity/get_statics
活动管理	查询活动列表	GET	/api/v1/activity/query
活动管理	更新活动信息	PUT	/api/v1/activity/update
志愿者管理	创建志愿者	POST	/api/v1/volunteer/create
志愿者管理	获取申请志愿者列表	GET	/api/v1/volunteer/list_application
志愿者管理	查询志愿者信息	GET	/api/v1/volunteer/query

模块名称	接口名称	请求方式	接口地址
首页	获取首页统计数据	GET	/api/v1/home/get_statics

2.2.5 人机界面设计

整体风格：采用浅橙色为主色调，界面布局规整、简洁明了，符合儿童阅读空间温馨、舒适的氛围，降低用户操作难度，提升用户体验感。

登录界面：提供手机号/账号、密码、验证码输入框，以及“登录”和“忘记密码”按钮，整体设计突出输入框和操作按钮，引导用户快速完成登录操作。

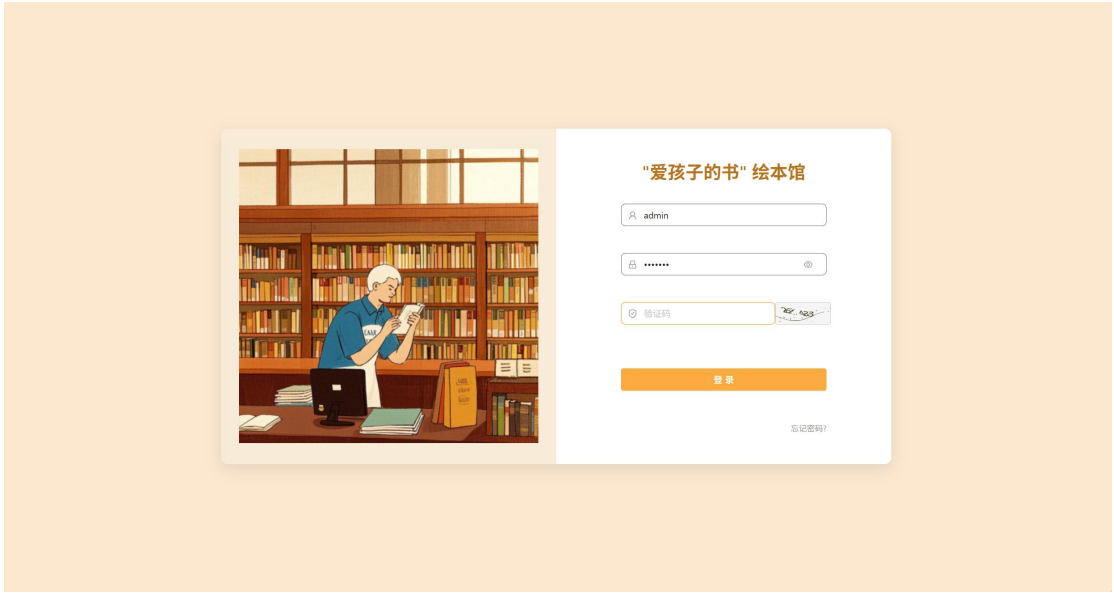


图 4 登录页面

各功能模块界面：每个功能模块界面都划分明确的区域，如库存管理模块分为信息检索区、操作功能区、库存信息展示区；会员管理模块分为信息检索区、会员信息操作区、会员统计区等，使用户能快速定位和操作所需功能。同时，各模块界面中的表格展示数据时，合理设置列宽和表头，确保数据清晰可读，并提供操作按钮，方便用户对数据进行处理。



图 5 首页页面

第三章详细设计

3.1 界面设计

登录页面：

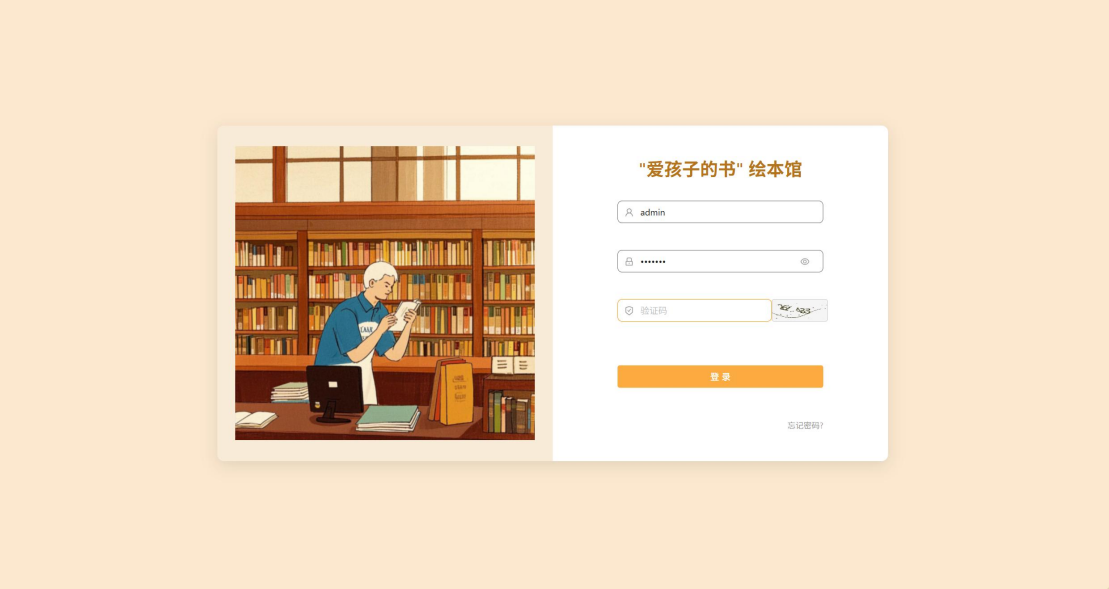


图 6 登录页面

首页：页面布局规整，分为**左侧导航栏、中间常用操作与数据展示区、消息通知区**。



图 7 首页页面

库存管理：界面布局清晰规整，上方为**信息检索区**，中间是**操作功能区**，下方是**库存信息展示区**。

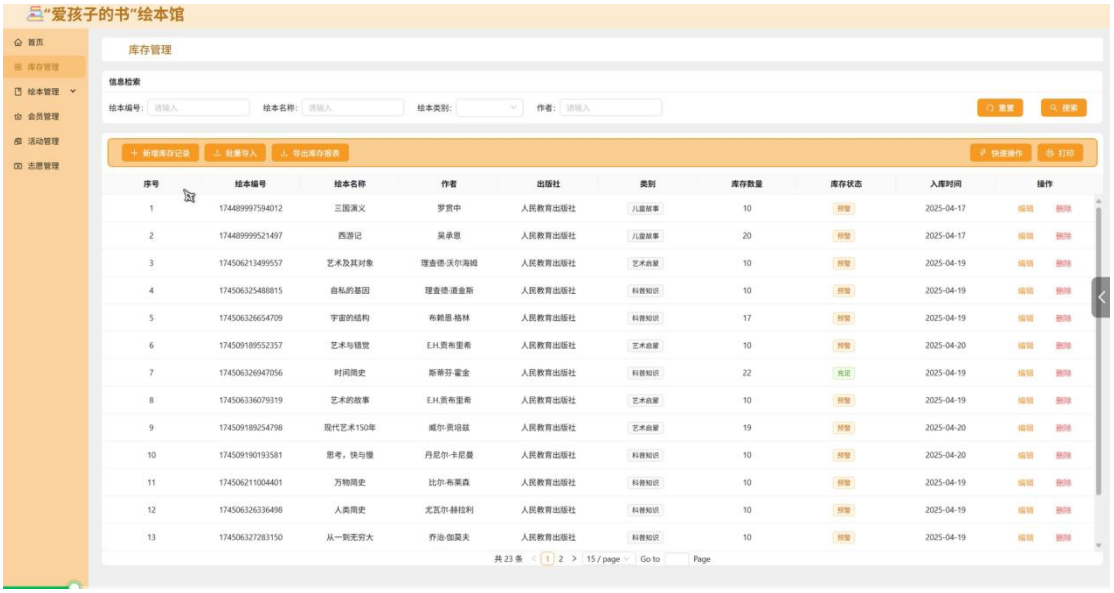


图 8 库存管理页面

会员管理：界面布局清晰，分为上方**信息检索区**、中部**会员信息操作区**以及右侧**会员统计区**。

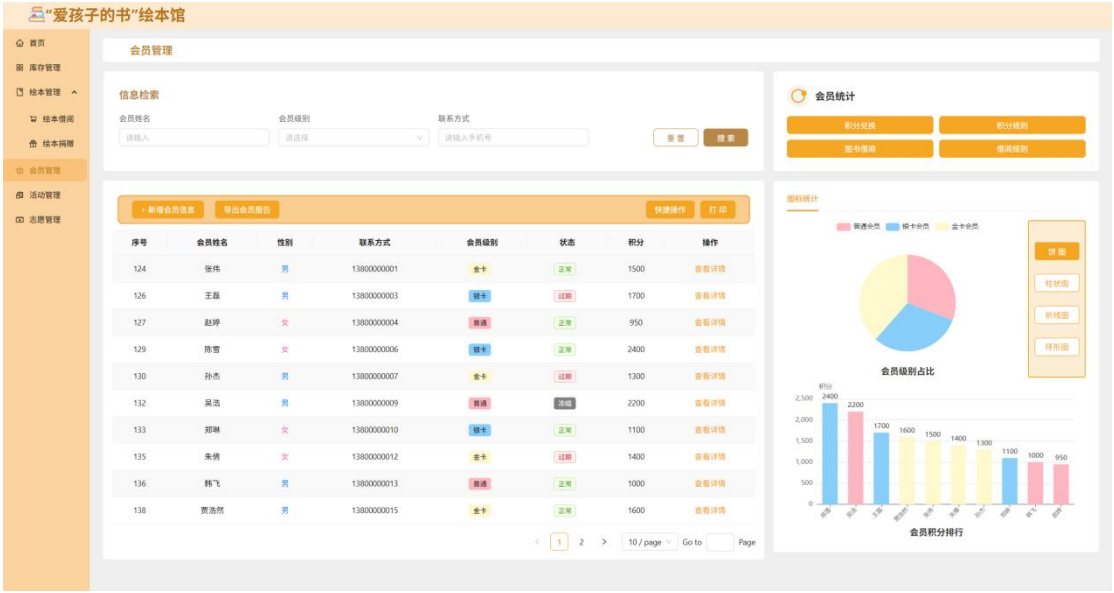


图 9 会员管理页面

活动管理：界面布局规整，分为**数据统计区**、**常用功能区**、**活动展示区**和**通知公告区**

通知公告：在页面右侧展示通知公告内容，管理员可点击“去发布”添加新公告，“更多”按钮可查看全部公告，帮助及时传达重要信息给参与者。

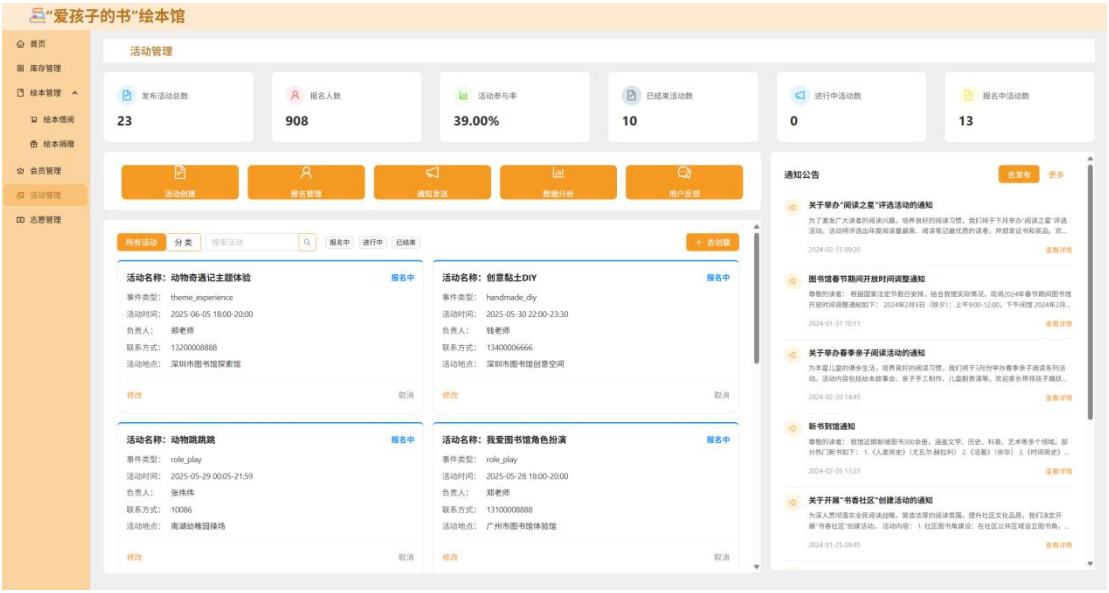


图 10 活动管理页面

志愿者管理：包括**志愿者信息、服务记录、志愿者招募、激励机制、数据/反馈**五大模板

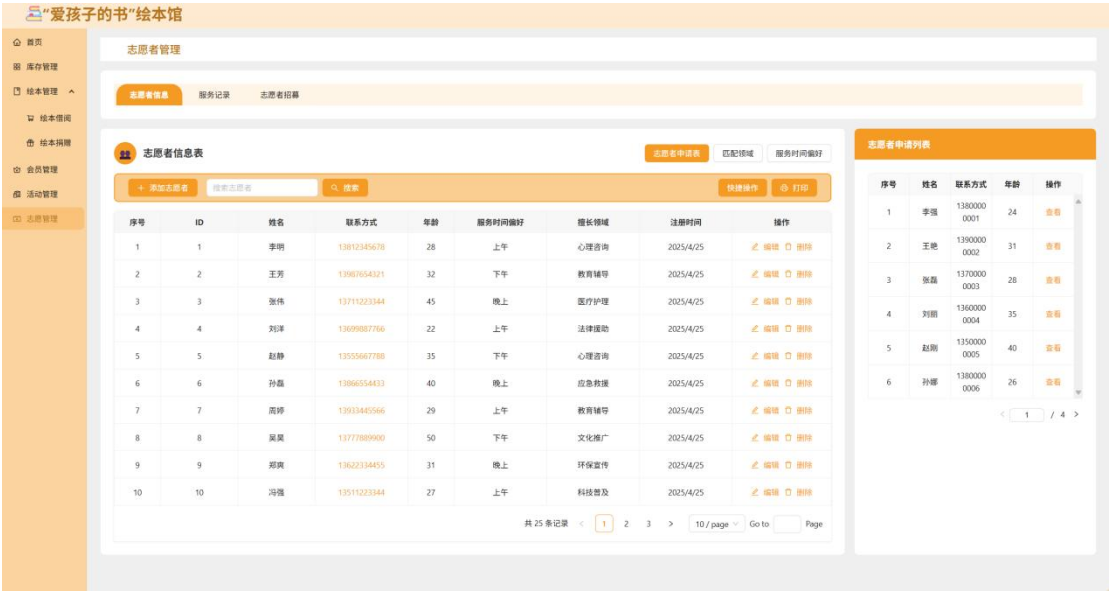


图 11 志愿者管理页面——志愿者信息

服务记录：

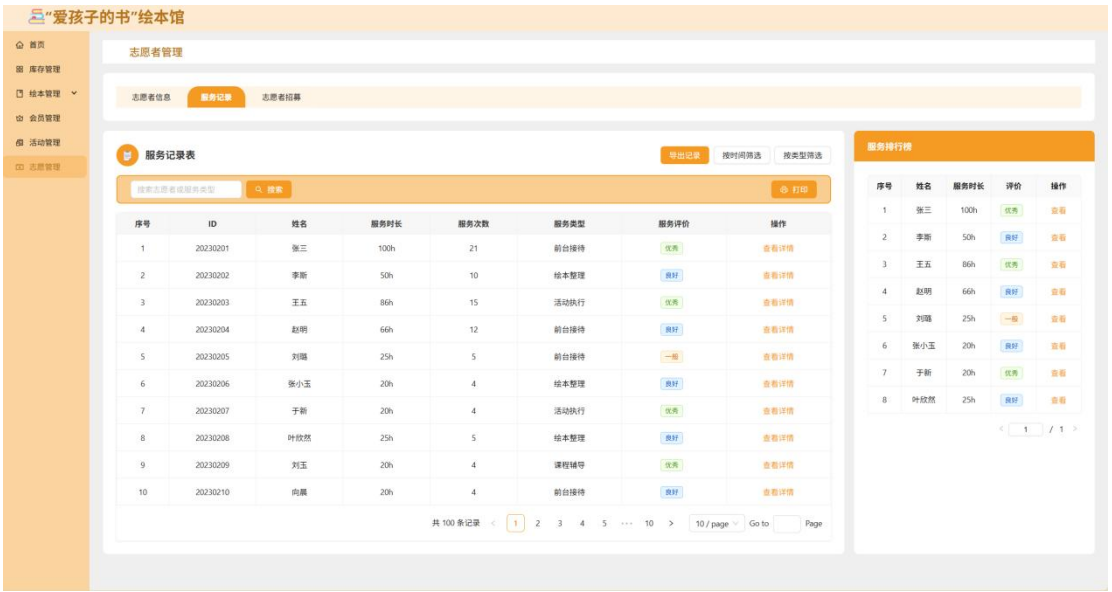


图 12 志愿者管理页面——服务记录

志愿者招募：

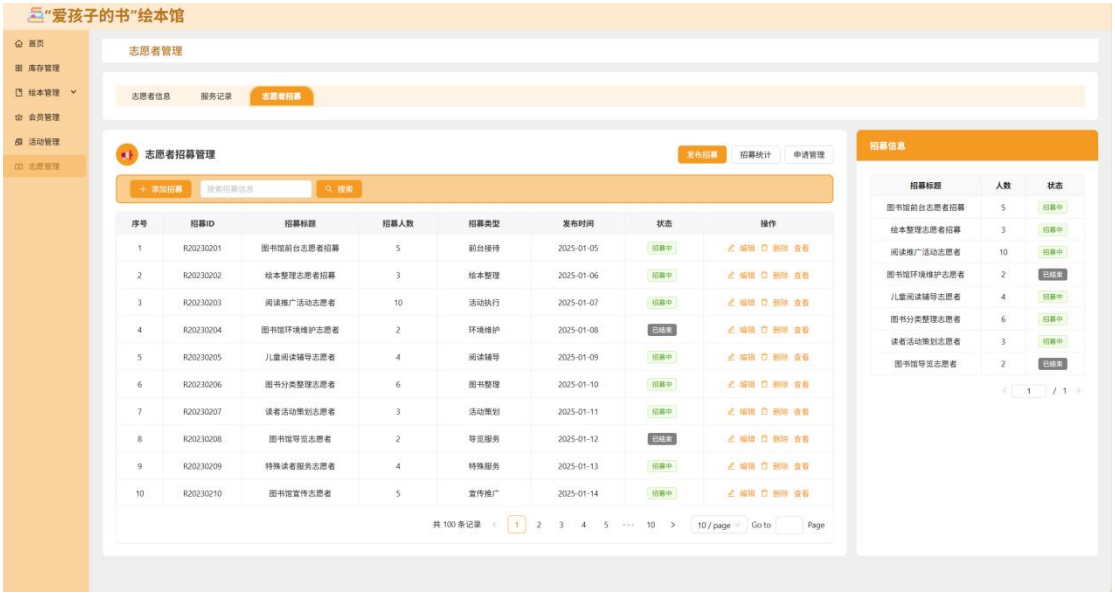


图 13 志愿者管理页面——志愿者招募

绘本管理：包括**绘本借阅、绘本捐赠**两大模块

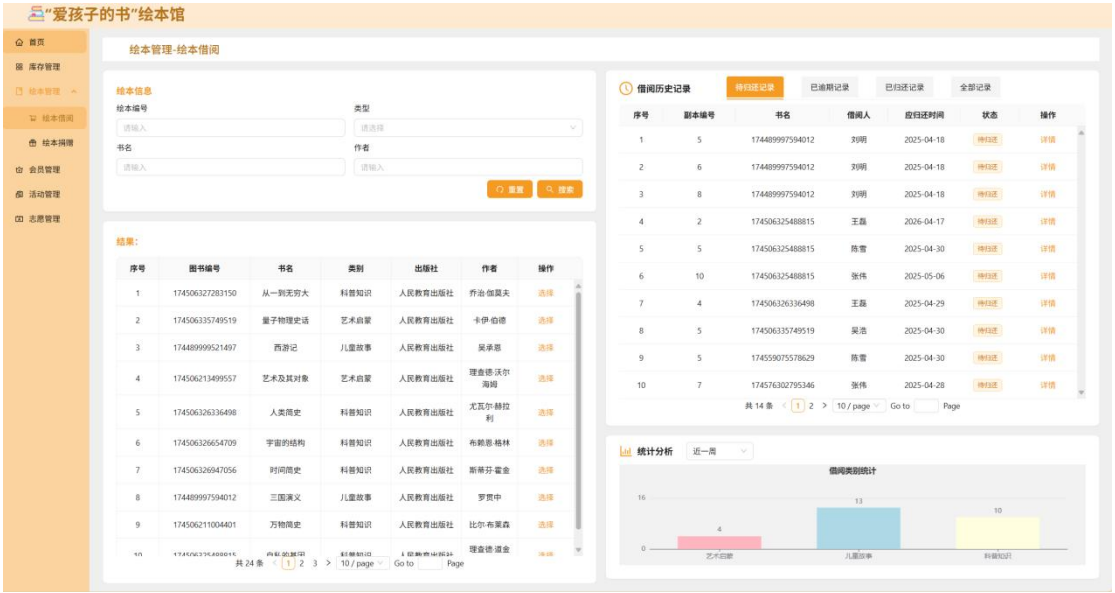


图 14 绘本管理——绘本借阅页面

绘本捐赠：左侧为**捐赠信息录入区**，右侧为**捐赠信息展示**和**统计区**。

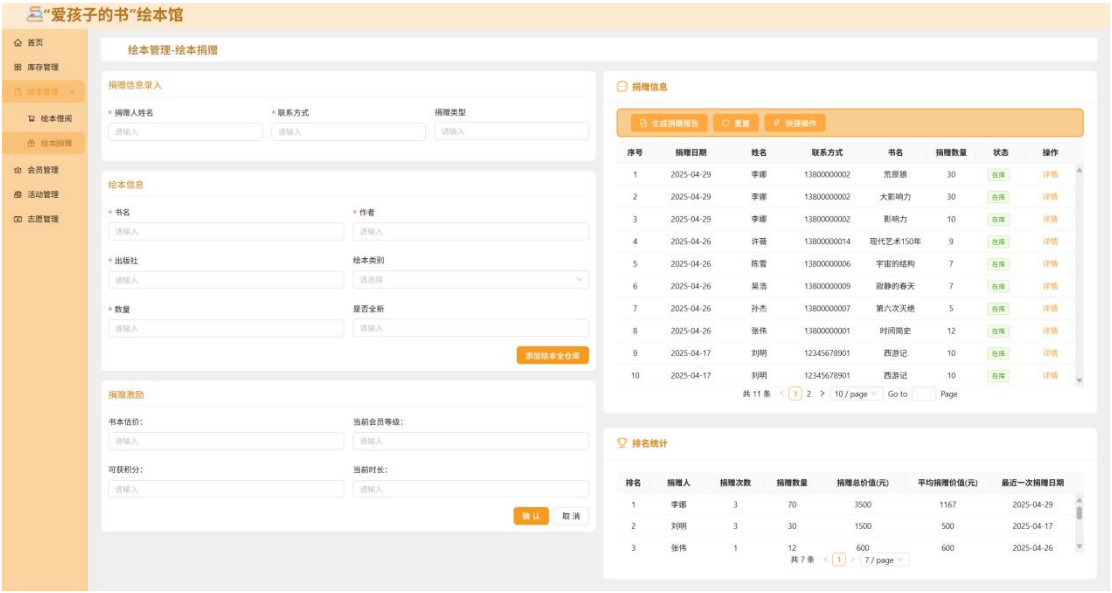


图 15 绘本管理——绘本捐赠页面

3.2 数据库设计

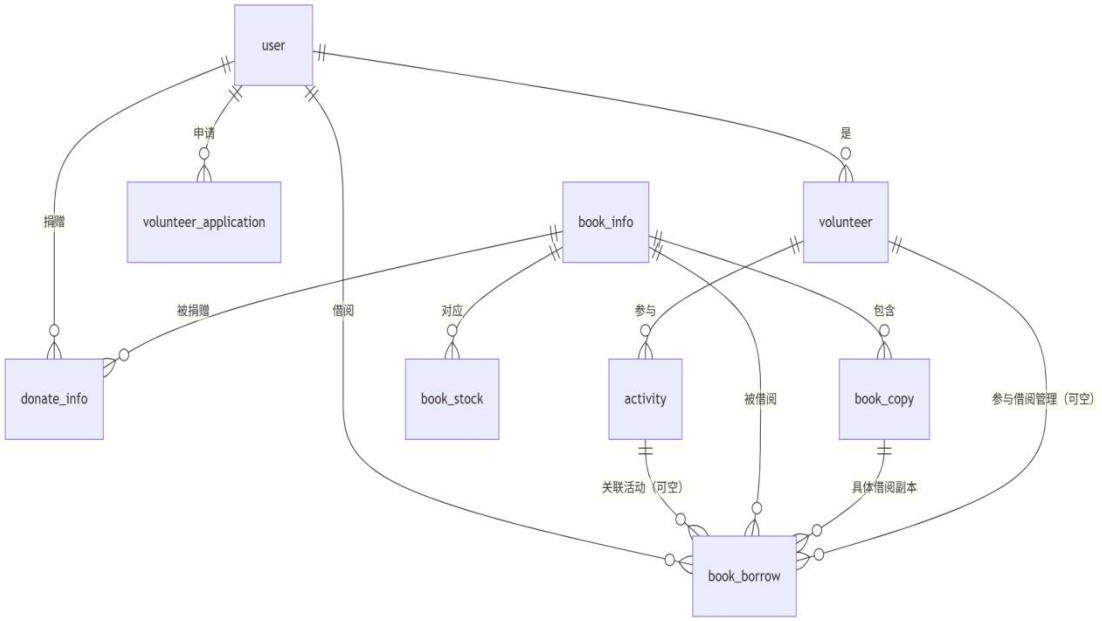


图 16 数据库实体 ER 图

1.user 表：存储用户信息的表，每个用户在表中对应一行记录。包括用户 ID（唯一标识用户，自增）、姓名、电话（唯一）、积分、性别、是否为 VIP、VIP 等级、状态以及用户创建时间等属性。

2.activity 表：存储活动信息的表，涵盖活动的名称、类型、开始时间、结束时间、负责人、联系电话、地址、预计人数、已报名人数等属性。每次活动在表中都有唯一对应的记录，方便管理和查询活动相关数据。

3.book_info 表：用于记录绘本的基本信息，如绘本 ID（主键）、名称、作者、出版社、类别、创建时间和更新时间。通过这些属性可以对绘本进行分类管理和检索。

4.book_stock 表：与 book_info 表关联，通过 book_id 建立外键关系。主要记录每种绘本的库存数量，以及库存信息的创建和更新时间，方便实时掌握绘本库存情况。

5.book_copy 表：保存绘本副本的相关信息，一本书可能有多个副本。通过 book_id 和 copy_id 唯一确定一个副本记录，同时记录副本的在库状态和是否为捐赠书等信息。
6.book_borrow 表：记录借阅行为的详细信息，通过 book_id 关联绘本信息，copy_id 确定具体借阅的副本，borrower_id 关联借阅用户。还包括预计归还时间、借阅时间、实际归还时间（可为空）和借阅状态等属性。
7.donate_info 表：存储捐赠信息，通过 user_id 和 book_id 分别关联用户和绘本，记录捐赠数量、创建时间和更新时间，用于统计捐赠情况。
8.volunteer 表：保存志愿者的详细信息，包括志愿者 ID（自增）、姓名、电话、年龄、服务时间偏好、擅长领域以及创建和更新时间，方便对志愿者进行管理和任务分配。
9.volunteer_application 表：记录志愿者申请信息，包含申请 ID、申请人姓名、电话、年龄、创建和更新时间，用于管理志愿者申请流程。

3.3 关键代码

后端开发使用 golang，前端开发使用 Typescript，开发框架基于 Next.js，采用 Ant Design 组件库保证界面统一性与交互友好性，集成 ECharts 可视化库实现多维度数据图表展示。后端基于 Gin 框架构建高性能 Web 服务，搭配 MySQL 数据库和 Redis 缓存中间件，通过 Docker 容器化部署实现高效运维；前端专注于数据可视化图表展示，提升用户交互体验。

后端核心代码如下：

```go
// Login 登录
// @Summary 登录
// @Description 登录
// @Tags 用户
// @Accept json
// @Produce json
// @Param object query LoginReq true "请求参数"
// @Success 200 {object} LoginResp "成功"
1 ref | move... |
func (a *AuthCtrl) Login(c *gin.Context) {

    var loginReq LoginReq
    if err := req.ParseRequestBody(c, &loginReq); err != nil {
        resp.SendResp(c, resp.NewRespFromErr(err))
        return
    }       You, 5天前 • feat(auth): 添加用户认证功能- 实现了用户登录和验证码生成接口 ……

    var (
        token string
        err   error
        val   string
    )

    if _, ok := a.up[loginReq.UserID]; !ok {
        resp.SendResp(c, resp.NewRespFromErr(errcode.UserNotFoundError))
        goto DelCode
    }
    if a.up[loginReq.UserID] != loginReq.Password {
        resp.SendResp(c, resp.NewRespFromErr(errcode.PasswordError))
        goto DelCode
    }
    val, err = a.rdb.Get(c, "captcha:"+loginReq.VerificationCodeID).Result()
    if err != nil {
        resp.SendResp(c, resp.NewRespFromErr(errcode.VerificationCodeError))
        goto DelCode
```

```go
no refs | impls 1 intfc method | move... |
func (b *BookBorrowRepo) GetBookStatisticsBorrow(ctx context.Context, pattern string, startTime, endTi
    //尝试从缓存中获取
    statistics, err := b.bookBorrowCache.GetBookBorrowStatistics(ctx, pattern)
    if err == nil {
        return statistics.ToMap(), nil
    }

    //这个操作，应该是个耗时操作
    if b.locker.IsLock() {
        logger.LogPrinter.Warnf("when book_repo is getting book borrow statistic, it has encountered a
        return nil, errors.New("locker: book_repo locker is lock")
    }
      You, 4周前 • refactor(internal): 重构图书相关接口和代码 …
    //加锁
    b.locker.Lock()
    defer b.locker.Unlock()

    statistics, err = b.bookBorrowDao.GetBookBorrowStatistics(ctx, startTime, endTime)
    if err == nil {
        err = b.bookBorrowCache.SaveBookBorrowStatistics(context.Background(), pattern, statistics)
        if err != nil {
            logger.LogPrinter.Warnf("cache: save book borrow statistics[pattern:%v statistics:%v] fail
        }
        return statistics.ToMap(), nil
    }
    //获取不到直接返回
    return nil, errors.New("get book borrow statistic failed")
}
```

func (b *BookStockRepo) getBookInID(ctx context.Context, ids ...uint64)
([]do.BookInfo, []do.BookStock, error) {
bookInfos, leftID := b.bookCache.GetBookInfoByID(ctx, ids...)
if len(leftID) > 0 {
tmpBookInfos, err := b.bookDao.GetBookInfoByID(ctx, leftID...)
if err != nil || len(tmpBookInfos) != len(leftID) {

```
return nil, nil, err
}
//异步保存书籍信息
defer func() {
go b.bookCache.SaveBookInfo(context.Background(),
tmpBookInfos...)
}()
bookInfos = append(bookInfos, tmpBookInfos...)
}
bookStocks, leftID := b.bookCache.GetBookStockByID(ctx, ids...)
if len(leftID) > 0 {
tmpBookStocks, err := b.bookDao.GetBookStockByID(ctx, leftID...)
if err != nil || len(tmpBookStocks) != len(leftID) {
return nil, nil, err
}
//异步保存书籍库存
defer func() {
go b.bookCache.SaveBookStock(context.Background(),
tmpBookStocks...)
}()
bookStocks = append(bookStocks, tmpBookStocks...)
}
return bookInfos, bookStocks, nil
}
161func (b *BookBorrowRepo) QueryBookRecord(ctx context.Context, pageSize int,
currentPage int, totalPage *int, opts ...func(db *gorm.DB)) ([]service.Boo
kBorrowRecord, error) {
if totalPage == nil || pageSize <= 0 || currentPage <= 0 {
return nil, errcode.PageError
}
num, err := b.bookBorrowDao.GetBookRecordTotalNum(ctx, opts...)
if err != nil {
return nil, err
}
*totalPage = tool.GetPage(num, pageSize)
if currentPage > *totalPage {
return nil, errcode.PageError
}
borrow, err := b.bookBorrowDao.FuzzyQueryBookBorrowRecord(ctx, pageSize, cu
rrentPage, opts...)
if err != nil {
return nil, err
}
var userID = make([]uint64, 0, len(borrow))
for _, v := range borrow {
userID = append(userID, v.BorrowerID)
```

```
}
mp, err := b.userDao.GetUserName(ctx, userID...)
if err != nil {
return nil, err
}
return batchToServiceBookRecord(borrow, mp), nil
}
func (b *BookSvc) QueryBookBorrowRecord(ctx context.Context, req controller.
QueryBookBorrowRecordReq, totalPage *int) ([]controller.BookBorrowRecord, e
rror) {
var opt func(db *gorm.DB)
if req.QueryStatus != nil {
switch *req.QueryStatus {
case common.BookStatusWaitingReturn, common.BookStatusOverdue, common.BookS
tatusReturned:
opt = func(db *gorm.DB) {
db.Where(fmt.Sprintf("%s.status = ?", common.BookBorrowTableName), *req.Que
ryStatus)
}
default:
return nil, errcode.ParamError
}
}
var (
records []BookBorrowRecord
err
error
)
if opt == nil {
records, err = b.bookBorrowRepo.QueryBookRecord(ctx, req.PageSize, req.Page,
totalPage)
} else {
records, err = b.bookBorrowRepo.QueryBookRecord(ctx, req.PageSize, req.Page,
totalPage, opt)
}
if err != nil {
return nil, errcode.SearchDataError
}
if req.Page > *totalPage {
return nil, errcode.PageError
}
return batchToControllerBookBorrowRecord(records), nil
}
```

前端用 AntDesign 封装的 Table 组件:

```
"use client";
```

```
import { useState } from "react";
import { Pagination, Table } from "antd";
import type { ColumnsType, TableProps } from "antd/es/table";
import type { PaginationProps } from "antd/es/pagination";
// currentPage pageSize 如果是外部传入 则表示是 受控 Table，dataSource 为当前页面的数据（需要外部切片数据），需要传入 onPageChange 函数，
// 否则 全部由默认处理 dataSource 为原始所有数据
// 其他参数用来定制 table 的常规配置
// columns 为 表格列配置
// Generic type for table data
export interface TangTableProps<T extends object = any> {
//具体看 antd
// 配置 table
// 表格列配置
columns: ColumnsType<T>;
// 数据数组
dataSource: T[];
// 表格是否可滚动，也可以指定滚动区域的宽、高
scroll?: { x?: number | string | true; y?: number | string };
// 表格大小
tableSize?: "large" | "middle" | "small";
// 配置 pagination
// Total number of items (for pagination)
total?: number;// Whether to show pagination
showPagination?: boolean;
// Custom pagination props
paginationProps?: PaginationProps;
// Controlled pagination values
currentPage?: number;
pageSize?: number;
// Pagination change handlers
onPageChange?: (page: number, pageSize: number) => void;
// Additional table props
tableProps?: Omit<
TableProps<T>,
"columns" | "dataSource" | "pagination" | "scroll" | "size"
>;
// Whether to show borders
bordered?: boolean;
}
export default function TangTable<T extends object = any>({
columns,
dataSource,
scroll,
tableSize = "small",
total,
```

```
showPagination = true,
paginationProps,
currentPage: externalCurrentPage,
pageSize: externalPageSize,
onPageChange,
tableProps,
bordered = false,
}: TangTableProps<T>) {
// Internal pagination state (used if not controlled externally)
const [internalCurrentPage, setInternalCurrentPage] = useState(1);
const [internalPageSize, setInternalPageSize] = useState(10);
// Determine if pagination is controlled externally
const isControlled =
externalCurrentPage !== undefined && externalPageSize !== undefined;
// Use either external or internal pagination values
const currentPage = isControlled ? externalCurrent
```

echart 封装的图表组件：

```
"use client"
import { useEffect, useRef } from "react"
import * as echarts from "echarts"
import { useResizeObserver } from "@/hooks/useResizeObserver"
const BorrowBarChart = () => {
const chartRef = useRef<HTMLDivElement>(null)
const chartInstance = useRef<echarts.ECharts | null>(null)
//  使用 ResizeObserver 监听容器大小变化
const { width, height } = useResizeObserver(chartRef)
164//  图表数据
const chartData = [
{ category: "故事类", value: 20 },
{ category: "科普类", value: 15 },
{ category: "益智类", value: 10 },
{ category: "童话类", value: 25 },
]
//  初始化图表
const initChart = () => {
if (!chartRef.current) return
//  创建或获取图表实例
if (!chartInstance.current) {
chartInstance.current = echarts.init(chartRef.current)
}
//  图表配置
const option = {
title: {
text: "本月借阅数量",
left: "center",
textStyle: {
```

```
color: "#333",
fontSize: 14,
},
},
tooltip: {
trigger: "axis",
axisPointer: {
type: "shadow",
},
},
grid: {
left: "3%",
right: "4%",
bottom: "3%",
containLabel: true,
},
xAxis: {
type: "category",
data: chartData.map((item) => item.category),
axisTick: {
alignWithLabel: true,
},
},
yAxis: {
type: "value",
min: 0,
max: 30,
interval: 5,
},
series: [
{
name: "借阅数量",
type: "bar",
barWidth: "60%",
data: chartData.map((item, index) => {
165// 为不同类别设置不同颜色
const colors = [
{ value: item.value, itemStyle: { color: "#ffb6c1" } }, // 故事类 - 粉色
{ value: item.value, itemStyle: { color: "#add8e6" } }, // 科普类 - 蓝色
{ value: item.value, itemStyle: { color: "#ffffe0" } }, // 益智类 - 黄色
{ value: item.value, itemStyle: { color: "#b0e0e6" } }, // 童话类 - 青色
]
```

```
return colors[index]
}),
},
],
}
// 设置图表选项
chartInstance.current.setOption(option)
}
// 当组件挂载时初始化图表
useEffect(() => {
initChart()
// 组件卸载时销毁图表
return () => {
if (chartInstance.current) {
chartInstance.current.dispose()
chartInstance.current = null
}
}
}, [])
// 当容器大小变化时重新调整图表大小
useEffect(() => {
if (width && height && chartInstance.current) {
chartInstance.current.resize()
}
}, [width, height])
return <div ref={chartRef} style={{height:'100%',width:'100%'}} />
}
export default BorrowBarChart
```

第四章测试报告

接口测试文档

一、简单测试用例

模块	接口名称	测试场景	输入数据	预期结果
认证	POST 登录	1. 正常登录	`{"user_id": "111", "password": "12345678", "verification_code_id": "abc123", "verification_code": "1234"}`	HTTP 200, 返回 `{"code": 200, "data": {"token": "..."}}`
		2. 密码错误	`{"user_id": "111", "password": "wrong", ...}`	HTTP 401, 返回 `{"code": 401, "msg": "密码错误"}`
	GET 获取图片验证码	1. 成功获取验证码	无参数	HTTP 200, 返回 `{"data": {"image": "base64字符串", "verification_code_id": "..."}}`
用户	GET 查询用户	1. 分页查询 VIP 用户	`?page=1&page_size=10&is_vip=true` (Header: `Authorization: Bearer xxx`)	HTTP 200, 返回分页数据且 `users[].is_vip` 均为 `true`
		2. 未授权访问	无 `Authorization` Header	HTTP 401
借书	POST 借书	1. 成功借书	`{"book_id": 1, "borrower_id": 111, "copy_id": 1, "expected_return_time": "2024-12-31"}`	HTTP 200, 返回 `{"data": {"book_id": 1, "copy_id": 1}}`

模块	接口名称	测试场景	输入数据	预期结果
		2. 借阅已借出的书籍	`{"book_id": 1, "copy_id": 1, ...}` (该副本已借出)	HTTP 400, 返回 `{"msg": "书籍不可借"}`
库存	POST 添加库存	1. 添加新书	`{"author": "鲁迅", "name": "呐喊", "category": "文学", "publisher": "人民文学", "quantity_added": 10}`	HTTP 200, 返回 `{"data": {"book_id": 新ID}}`
		2. 必填字段缺失	`{"name": "呐喊"}` (缺少 author/category)	HTTP 400, 返回 `{"msg": "缺少必填字段"}`
活动管理	POST 新增活动	1. 创建有效活动	`{"info": {"activity_name": "读书会", "start_time": "2024-10-01", "end_time": "2024-10-02", ...}}`	HTTP 200, 返回 `{"data": {"activity_id": 1}}`
		2. 结束时间早于开始时间	`{"info": {"start_time": "2024-10-02", "end_time": "2024-10-01", ...}}`	HTTP 400, 返回 `{"msg": "时间无效"}`
志愿者管理	POST 创建志愿者	1. 正常创建	`{"name": "张三", "phone": "13800138000", "age": 25, "expertiseArea": "教育"}`	HTTP 200, 返回 `{"data": {"volunteer_id": 1}}`

模块	接口名称	测试场景	输入数据	预期结果
		2. 手机号格式错误	`{"phone": "123", ...}`	HTTP 400, 返回 `{"msg": "手机号无效"}`
首页	GET 获取首页统计数据	1. 验证数据完整性	Header: `Authorization: Bearer xxx`	HTTP 200, 返回 `{"total_stock": 100, "today_borrowed": 5, ...}`

二、响应速度要求

接口分类	主要接口路径	最大响应时间（90%请求）	最大响应时间（极端）
登录相关	/api/v1/auth/login, /api/v1/auth/get_verification_code	≤500ms	≤1000ms
用户相关	/api/v1/user/**	≤800ms	≤1500ms
借书、库存相关	/api/v1/book/**	≤1000ms	≤2000ms
活动、捐赠、志愿者管理	/api/v1/activity/, **/api/v1/book/donate/**, /api/v1/volunteer/**	≤1200ms	≤2000ms

说明：

- 90%的接口响应应满足"最大响应时间"以内；
- 极端情况下（如服务器高峰期）允许单次请求稍超，但需在监控中及时告警。

三、部署方式

使用dockerfile打包，并使用docker-compose进行一键部署，并使用CI/CD实现自动化部署

本项目采用Docker技术进行容器化部署。首先，通过编写Dockerfile对应用进行打包，确保应用及其依赖环境一致，提升了部署的稳定性与可移植性。随后，使用docker-compose对多个服务进行统一管理，实现一键部署，简化了复杂应用的启动流程，极大提高了部署效率。

为了进一步优化部署流程，本项目还集成了CI/CD（持续集成/持续交付）机制。当代码推送到版本控制系统后，CI/CD流水线会自动完成代码构建、镜像生成、推送至镜像仓库，并触发远程服务器的自动部署，实现了从代码提交到上线的全流程自动化，减少了人为操作带来的失误，同时也加快了开发迭代速度。

通过以上部署方式，系统具备了快速部署、版本可控、回滚便捷等优点，为项目的稳定运行和后续扩展提供了坚实保障。

第五章安装及使用

该图书管理系统集图书借阅、捐赠跟踪和活动管理功能于一体。安装环境涉及多种软件，安装过程分前后端进行，使用流程涵盖登录与后台操作。

5.1 安装环境

后端：需安装 Go1.23+作为主要编程语言运行环境，MySQL5.7+作为主要关系型数据库，Redis6+用于性能优化缓存层。若进行容器化部署，还需安装 Docker 和 Docker Compose；此外，开发过程中涉及 Wire（依赖注入）、Swagger（API 文档生成）、Makefile（构建自动化）、Go Test（单元测试框架）、GitHub Actions（CI/CD 流水线）等工具。

前端：要安装并配置 Git（参考版本 2.46.1.windows.1 ）和 Nodejs（版本 v22.11.0），推荐安装 pnpm 包管理器（也可使用 npm）。操作系统支持 Windows 和 MacOS，浏览器需为 chromium 内核，如 Chrome、Edge。

5.2 安装过程

后端：本地开发时，先在项目目录执行`go mod download`下载依赖；接着执行 make swagger 生成 Swagger 文档；最后运行`go run cmd/main.go -conf configs/config.yaml 启动应用。使用 Docker 部署时，先在项目根目录执行 docker build -t bm.`构建镜像，再执行 docker-compose up 启动服务。

前端：从网上下载安装，在系统终端（Windows 的 pws 或 cmd、macOS 的 zsh ）中输入 gitclonegit@github.com:drunksweet/picbook-fe.git， cd picbook-fe ; npm install;npm dev（Windows 的 pws ）或 git clonegit@github.com:drunksweet/picbook-fe.git && cd picbook-fe && npm install && npm dev（macOS 的 zsh ），等待克隆和安装完成后，在本地浏览器访问 localhost:3000。若使用安装包安装，先解压 picbook-fe-main.zip 文件，在解压后的根目录下的终端执行 npm install ; npm dev，完成后在浏览器访问 localhost:3000。

5.3 主要流程：

登录注册：使用官方测试账号登录，账户名“admin” ，密码“12345678”，输入验证码后点击登录，自动跳转到首页。 - **绘本后台操作**：登录后，通过右侧导航进行页面跳转，依据页面提示对图书、志愿者、活动、会员等数据执行增删查改操作以及数据可视化操作。比如在图书管理模块，可通过`/api/books`获取所有图书信息，通过`/api/books/borrow`借阅图书；在活动管理模块，可通过`/api/activities`获取所有活动信息；在捐赠管理模块，可通过`/api/donations`进行图书捐赠操作。

第六章项目总结

6.1 项目协调

团队围绕儿童绘本馆智慧运营管理系统展开深入探讨，旨在打造一个集绘本借阅、活动管理、会员服务等多功能于一体的综合性平台。初期，我们研究了市场上现有的绘本馆运营模式，分析用户需求，确定以提升用户体验、优化管理效率为核心目标。随后制定项目计划，明确各阶段里程碑和交付物，如完成系统架构设计、实现各功能模块开发、进行全面测试等。根据成员专长分配任务，涵盖前端开发、后端编程、数据库设计、测试等方面，并建立每周团队会议制度，及时沟通项目进展、协调资源、解决问题，确保项目按计划推进。

6.2 任务分解

依据系统功能模块，将项目拆解为多个子任务。前端负责设计友好舒适的用户界面，包括登录页面、首页、各功能操作页面等，运用 HTML、CSS、JavaScript 等技术实现页面交互效果；后端利用 Go 语言结合 Gin 框架搭建 Web 服务，通过 GORM 操作 MySQL 数据库，完成用户认证、数据存储与读取等业务逻辑；数据库设计则根据系统需求构建用户、绘本、活动等数据表，确定字段类型、约束及表间关系。每个子任务由 1 名成员主负责，定期汇报任务进度，团队共同审查成果，保障各部分工作质量与整体项目的协同性。

6.3 克服困难

项目推进中面临诸多挑战。技术选型时，在众多数据库和框架中抉择困难，需综合考量性能、易用性、社区支持等因素。开发过程里，前后端数据交互、接口对接问题频发，如数据格式不一致、接口参数不匹配等。例如，在实现绘本借阅功能时，借阅记录的保存和查询出现数据错误。通过深入分析问题根源，查阅技术文档，团队成员相互协作，优化数据传输格式、完善接口定义，成功解决问题。此外，系统性能优化也是难题，高并发下响应速度慢，经数据库索引优化、缓存机制引入，显著提升系统性能。

6.4 升级演进

后续将分阶段升级系统。近期聚焦功能优化，完善用户反馈机制，根据用户意见改进操作流程；优化库存管理模块，实现库存自动预警和补货提醒。中期拓展功能，开发亲子互动功能，如线上亲子阅读打卡、阅读心得分享；利用数据分析技术，为会员精准推荐绘本。远期进行技术架构升级，引入微服务架构提升系统扩展性和维护性；开发移动端应用，支持 IOS 和 Android 系统，方便用户随时随地使用绘本馆服务，同时拓展商业推广，与周边学校、幼儿园合作，扩大用户群体。

参考文献

[1] 张毅红.儿童哲学与儿童阅读疗愈的融合与应用研究[J].图书馆杂志,2025,44(04):42-47+122.DOI:10.13663/j.cnki.lj.2025.04.005.

[2] 朱美华.公共图书馆体系化高质量开展绘本阅读推广的路径分析——基于东莞图书馆“绘本馆体系化建设项目”的思考[J/OL].图书馆杂志,1-19[2025-05-02].http://kns.cnki.net/kcms/detail/31.1108.G2.20250113.0903.002.html.

[3] 彭雨.儿童阅读App的互动性叙事与儿童主体性建构[J].四川师范大学学报(社会科学版),2024,51(05):133-141+205.DOI:10.13734/j.cnki.1000-5315.2024.0614.

[4] 司亚丽,王泽艺,贾丽.基于知识图谱的国内外儿童绘本可视化研究[J].包装工程,2024,45(14):308-323+344.DOI:10.19554/j.cnki.1001-3563.2024.14.032.

[5] 牟艾莉.设计教育融入儿童绘本的创新探索与实践——以“超级游戏力”系列绘本为例[J].装饰,2024,(01):142-144.DOI:10.16272/j.cnki.cn11-1392/j.2024.01.020.

[6] Werchan M D ,Aitken B A ,Ku S , et al.Naturalistic patterns of sustained attention across early childhood: Developmental profiles and longitudinal associations with executive functions.[J].Developmental psychology,2024,

[7] Mathers J S ,Hodgkiss A ,Kolancali P , et al.Comparing parent-child interaction during wordless book reading, print book reading and imaginative play.[J].Journal of child language,2024,21-26.

[8] J. D P ,D. C O ,Alexa E , et al.Unique and combined effects of quantitative mathematical language and numeracy instruction within a picture book intervention: A registered report.[J].Journal of Educational Psychology,2024,116(1):1-19.

[9] 谢子杰.巧用儿童绘本激活审美创造——绘本与幼儿美育课程融合的策略探索[J].文科爱好者,2025,(03):242-244.

[10] 陈雨轩,尹毅.幼儿敏感期的儿童绘本设计解析[J].丝网印刷,2024,(19):47-49.DOI:10.20084/j.cnki.1002-4867.2024.19.014.